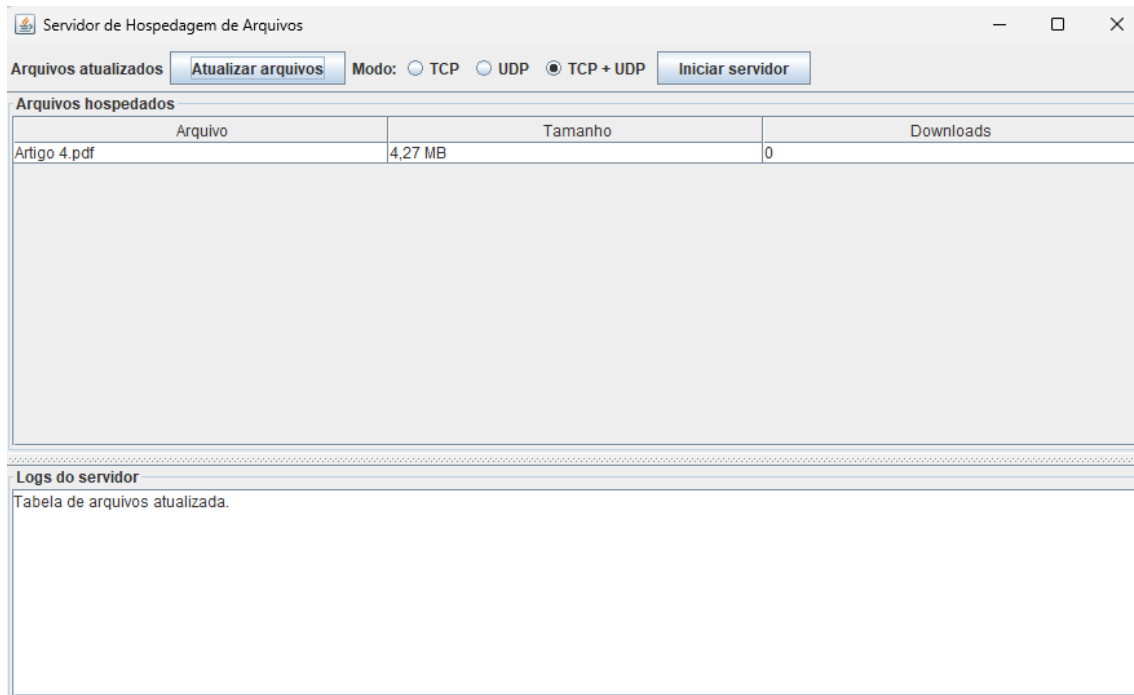


Curso Técnico em Desenvolvimento de Sistemas

Unidade Curricular: Desenvolvimento de Sistemas

Docente: Gustavo Garcia de Amo

Servidor de hospedagem de arquivos. Resultado esperado:



Desenvolva um sistema cliente-servidor em Java capaz de simular um servidor de hospedagem de arquivos.

O servidor deve:

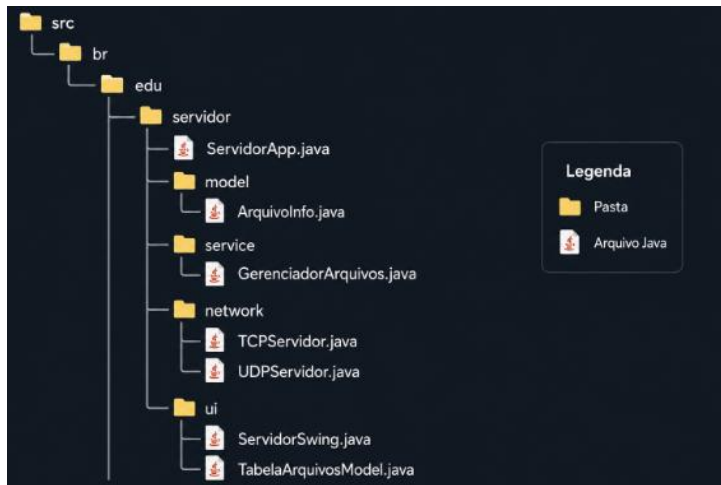
- Ler os arquivos de um diretório local;
- Exibir os arquivos em uma interface Swing (`JTable`);
- Mostrar:
 - nome do arquivo;
 - tamanho;
 - quantidade de downloads;
- Utilizar:
 - `EDT (Event Dispatch Thread)`;
 - `SwingWorker`;
 - comunicação `TCP`;
 - comunicação `UDP`.

Funcionalidades obrigatórias:

1. O servidor deve responder via `TCP`:
 - listagem de arquivos;
 - download de arquivos.
2. O servidor deve responder via `UDP`:

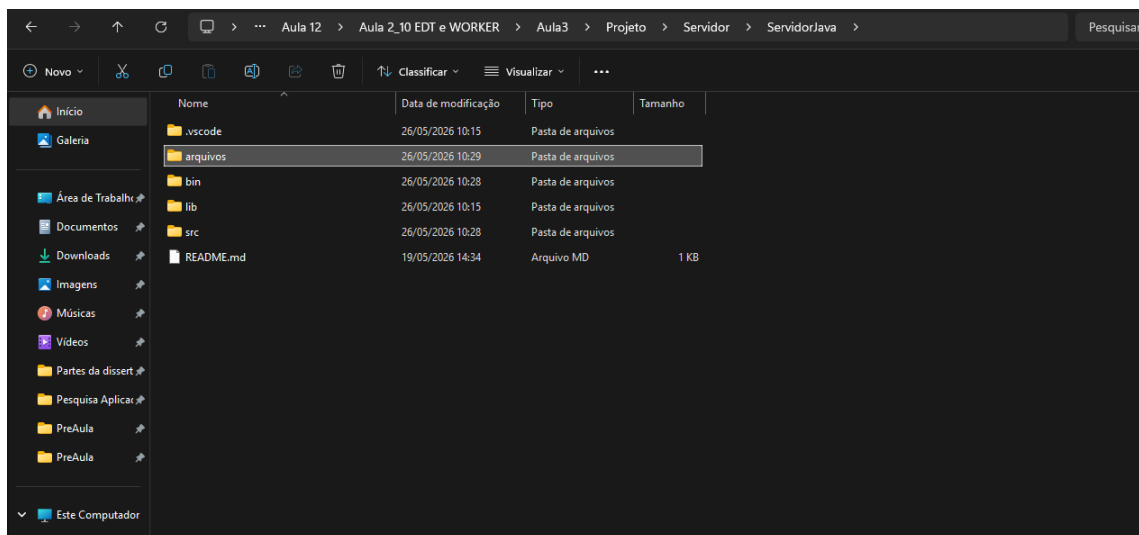
- descoberta do servidor;
 - listagem rápida de arquivos.
3. O cliente deve:
- conectar ao servidor;
 - listar arquivos disponíveis;
 - realizar download.
4. O servidor deve atualizar automaticamente a quantidade de downloads na interface gráfica.

Organize o projeto em classes e pacotes. Estrutura para o SERVIDOR:



Instruções de Implementação

1- Crie uma pasta dentro do projeto para armazenar os arquivos



2- Implementar a classe: ServidorApp.java

```
package br.edu.servidor;

import br.edu.servidor.ui.ServidorSwing;
```

```

import javax.swing.SwingUtilities;

public class ServidorApp {

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {

            ServidorSwing tela = new ServidorSwing();

            tela.setVisible(true);

        });

    }

}

```

3- Criar e implementar a classe modelo. ArquivoInfo.java

nome do arquivo

tamanho do arquivo

quantidade de downloads

```

package br.edu.servidor.modelo;

public class ArquivoInfo {

    private String nome;

    private long tamanho;

    private int downloads;

    public ArquivoInfo(String nome, long tamanho, int downloads) {

        this.nome = nome;

        this.tamanho = tamanho;

        this.downloads = downloads;

    }

    public String getNome() {

        return nome;

    }

    public long getTamanho() {

        return tamanho;

    }

    public int getDownloads() {

        return downloads;

    }

}

```

```

    }

    public void incrementarDownloads() {
        downloads++;
    }

    public String getTamanhoFormatado() {
        if (tamanho < 1024) {
            return tamanho + " B";
        } else if (tamanho < 1024 * 1024) {
            return String.format("%.2f KB", tamanho / 1024.0);
        } else {
            return String.format("%.2f MB", tamanho / (1024.0 * 1024.0));
        }
    }
}

```

4- Criar o gerenciador de arquivos. GerenciadorArquivos.java.

Ações da classe:

ler os arquivos de uma pasta chamada arquivos

listar os arquivos encontrados

buscar um arquivo pelo nome

incrementar o número de downloads

```

package br.edu.servidor.service;

import br.edu.servidor.model.ArquivoInfo;
import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;

public class GerenciadorArquivos {
    private final File diretorio;
    private final ConcurrentHashMap<String, Integer> contadorDownloads;
}

```

```

public GerenciadorArquivos(String caminhoDiretorio) {
    this.diretorio = new File(caminhoDiretorio);
    this.contadorDownloads = new ConcurrentHashMap<>();

    if (!diretorio.exists()) {
        diretorio.mkdirs();
    }
}

public synchronized List<ArquivoInfo> listarArquivos() {
    List<ArquivoInfo> lista = new ArrayList<>();

    File[] arquivos = diretorio.listFiles();

    if (arquivos != null) {
        for (File arquivo : arquivos) {
            if (arquivo.isFile()) {
                int downloads =
contadorDownloads.getOrDefault(arquivo.getName(), 0);

                lista.add(new ArquivoInfo(
                    arquivo.getName(),
                    arquivo.length(),
                    downloads
                ));
            }
        }
    }

    return lista;
}

public synchronized File buscarArquivo(String nomeArquivo) {
    File arquivo = new File(diretorio, nomeArquivo);

```

```

        if (arquivo.exists() && arquivo.isFile()) {
            return arquivo;
        }

        return null;
    }

    public synchronized void incrementarDownload(String nomeArquivo) {
        contadorDownloads.put(
            nomeArquivo,
            contadorDownloads.getOrDefault(nomeArquivo, 0) + 1
        );
    }

    public String gerarListaTexto() {
        StringBuilder sb = new StringBuilder();

        for (ArquivoInfo arquivo : listarArquivos()) {
            sb.append(arquivo.getNome())
              .append(";")
              .append(arquivo.getTamanho())
              .append(";")
              .append(arquivo.getDownloads())
              .append("\n");
        }

        return sb.toString();
    }
}

```

5- Criar a tabela. TabelaArquivosModel.java

Estender o AbstractTableModel. Colunas: Arquivo, tamanho e Downloads

```
package br.edu.servidor.ui;

import br.edu.servidor.model.ArquivoInfo;
import javax.swing.table.AbstractTableModel;
import java.util.ArrayList;
import java.util.List;

public class TabelaArquivosModel extends AbstractTableModel {

    private final String[] colunas = {"Arquivo", "Tamanho", "Downloads"};
    private List<ArquivoInfo> arquivos = new ArrayList<>();

    public void setArquivos(List<ArquivoInfo> arquivos) {
        this.arquivos = arquivos;
        fireTableDataChanged();
    }

    @Override
    public int getRowCount() {
        return arquivos.size();
    }

    @Override
    public int getColumnCount() {
        return colunas.length;
    }

    @Override
    public String getColumnName(int column) {
        return colunas[column];
    }

    @Override
```

```

    public Object getValueAt(int rowIndex, int columnIndex) {
        ArquivoInfo arquivo = arquivos.get(rowIndex);

        return switch (columnIndex) {
            case 0 -> arquivo.getNome();
            case 1 -> arquivo.getTamanhoFormatado();
            case 2 -> arquivo.getDownloads();
            default -> "";
        };
    }
}

```

6- Criar a tela do servidor. SwingUtilities.invokeLater(...). ServidorSwing.java

JTable para arquivos

JTextArea para logs

JButton para atualizar

JLabel para status

```

package br.edu.servidor.ui;

import br.edu.servidor.network.TCPServidor;
import br.edu.servidor.network UDPServidor;
import br.edu.servidor.service.GerenciadorArquivos;

import javax.swing.*;
import java.awt.*;

public class ServidorSwing extends JFrame {

    private JTable tabelaArquivos;
    private JTextArea areaLogs;
    private JButton btnAtualizar;
    private JButton btnIniciarServidor;
    private JLabel lblStatus;

```



```
private JRadioButton radioTCP;
private JRadioButton radioUDP;
private JRadioButton radioAmbos;

private TabelaArquivosModel tabelaModel;
private GerenciadorArquivos gerenciador;

private TCPServidor tcpServidor;
private UDPServidor udpServidor;

private static final int PORTA_TCP = 5000;
private static final int PORTA_UDP = 6000;

private boolean servidorIniciado = false;

public ServidorSwing() {
    setTitle("Servidor de Hospedagem de Arquivos");
    setSize(900, 550);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    gerenciador = new GerenciadorArquivos("arquivos");

    inicializarComponentes();
    carregarArquivos();
}

private void inicializarComponentes() {
    tabelaModel = new TabelaArquivosModel();
    tabelaArquivos = new JTable(tabelaModel);

    areaLogs = new JTextArea();
```

```
areaLogs.setEditable(false);

btnAtualizar = new JButton("Atualizar arquivos");
btnIniciarServidor = new JButton("Iniciar servidor");

lblStatus = new JLabel("Servidor ainda não iniciado");

radioTCP = new JRadioButton("TCP");
radioUDP = new JRadioButton("UDP");
radioAmbos = new JRadioButton("TCP + UDP", true);

ButtonGroup grupoProtocolos = new ButtonGroup();
grupoProtocolos.add(radioTCP);
grupoProtocolos.add(radioUDP);
grupoProtocolos.add(radioAmbos);

JPanel painelTopo = new JPanel(new FlowLayout(FlowLayout.LEFT));

painelTopo.add(lblStatus);
painelTopo.add(btnAtualizar);
painelTopo.add(new JLabel("Modo:"));
painelTopo.add(radioTCP);
painelTopo.add(radioUDP);
painelTopo.add(radioAmbos);
painelTopo.add(btnIniciarServidor);

JPanel painelTabela = new JPanel(new BorderLayout());
painelTabela.setBorder(BorderFactory.createTitledBorder("Arquivos  
hospedados"));
painelTabela.add(new JScrollPane(tabelaArquivos),  
BorderLayout.CENTER);

JPanel painelLogs = new JPanel(new BorderLayout());
painelLogs.setBorder(BorderFactory.createTitledBorder("Logs do  
servidor"));
```

```

painelLogs.add(new JScrollPane(areaLogs), BorderLayout.CENTER);

JSplitPane splitPane = new JSplitPane(
    JSplitPane.VERTICAL_SPLIT,
    painelTabela,
    painelLogs
);

splitPane.setDividerLocation(280);

add(painelTopo, BorderLayout.NORTH);
add(splitPane, BorderLayout.CENTER);

btnAtualizar.addActionListener(e -> carregarArquivos());
btnIniciarServidor.addActionListener(e -> iniciarServidores());
}

private void carregarArquivos() {
    SwingWorker<Void, Void> worker = new SwingWorker<>() {

        @Override
        protected Void doInBackground() {
            tabelaModel.setArquivos(gerenciador.listarArquivos());
            return null;
        }

        @Override
        protected void done() {
            lblStatus.setText("Arquivos atualizados");
            adicionarLog("Tabela de arquivos atualizada.");
        }
    };
};

```

```

        worker.execute();
    }

    private void iniciarServidores() {
        if (servidorIniciado) {
            JOptionPane.showMessageDialog(
                this,
                "O servidor já foi iniciado. Para mudar o modo, feche e abra novamente."
            );
            return;
        }

        if (radioTCP.isSelected()) {
            iniciarTCP();
            lblStatus.setText("Servidor iniciado somente em TCP");
        } else if (radioUDP.isSelected()) {
            iniciarUDP();
            lblStatus.setText("Servidor iniciado somente em UDP");
        } else {
            iniciarTCP();
            iniciarUDP();
            lblStatus.setText("Servidor iniciado em TCP + UDP");
        }

        servidorIniciado = true;

        radioTCP.setEnabled(false);
        radioUDP.setEnabled(false);
        radioAmbos.setEnabled(false);
        btnIniciarServidor.setEnabled(false);
    }

```

```

private void iniciarTCP() {
    tcpServidor = new TCPServidor(
        PORTA_TCP,
        gerenciador,
        this::adicionarLog,
        this::carregarArquivos
    );

    SwingWorker<Void, Void> workerTCP = new SwingWorker<>() {

        @Override
        protected Void doInBackground() {
            tcpServidor.iniciar();
            return null;
        }
    };

    workerTCP.execute();

    adicionarLog("Servidor TCP iniciado na porta " + PORTA_TCP);
}

private void iniciarUDP() {
    udpServidor = new UDPServidor(
        PORTA_UDP,
        gerenciador,
        this::adicionarLog
    );

    SwingWorker<Void, Void> workerUDP = new SwingWorker<>() {

        @Override

```

```

        protected Void doInBackground() {
            udpServidor.iniciar();
            return null;
        }
    };

    workerUDP.execute();

    adicionarLog("Servidor UDP iniciado na porta " + PORTA_UDP);
}

private void adicionarLog(String mensagem) {
    SwingUtilities.invokeLater(() -> {
        areaLogs.append(mensagem + "\n");
    });
}
}
}

```

7- Usar SwingWorker. Criar a Classe TCPServidor

carregar arquivos sem travar a tela

iniciar o servidor TCP

iniciar o servidor UDP

```

package br.edu.servidor.network;

import br.edu.servidor.service.GerenciadorArquivos;

import javax.swing.SwingUtilities;
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.function.Consumer;

```

```
public class TCPServidor {

    private final int porta;
    private final GerenciadorArquivos gerenciador;
    private final Consumer<String> log;
    private final Runnable aoAtualizarTabela;

    private boolean executando = false;

    public TCPServidor(
        int porta,
        GerenciadorArquivos gerenciador,
        Consumer<String> log,
        Runnable aoAtualizarTabela
    ) {
        this.porta = porta;
        this.gerenciador = gerenciador;
        this.log = log;
        this.aoAtualizarTabela = aoAtualizarTabela;
    }

    public void iniciar() {
        executando = true;

        try (ServerSocket serverSocket = new ServerSocket(porta)) {
            log.accept("Servidor TCP iniciado na porta " + porta);

            while (executando) {
                Socket socket = serverSocket.accept();

                new Thread(() -> atenderCliente(socket)).start();
            }
        }
    }
}
```

```

    } catch (IOException e) {
        log.accept("Erro no servidor TCP: " + e.getMessage());
    }
}

private void atenderCliente(Socket socket) {
    try (
        DataInputStream entrada = new
DataInputStream(socket.getInputStream());
        DataOutputStream saida = new
DataOutputStream(socket.getOutputStream())
    ) {
        String comando = entrada.readUTF();

        if (comando.equalsIgnoreCase("LIST")) {
            enviarLista(saida);
        } else if (comando.startsWith("DOWNLOAD")) {
            String nomeArquivo = comando.replace("DOWNLOAD",
""").trim();
            enviarArquivo(nomeArquivo, saida);
        } else {
            saida.writeUTF("ERRO Comando inválido");
        }
    }

    } catch (IOException e) {
        log.accept("Erro ao atender cliente: " + e.getMessage());
    }
}

private void enviarLista(DataOutputStream saida) throws IOException {
    String lista = gerenciador.gerarListaTexto();

    saida.writeUTF("OK");
    saida.writeUTF(lista);
}

```



```

        log.accept("Lista de arquivos enviada para cliente TCP.");
    }

    private void enviarArquivo(String nomeArquivo, DataOutputStream saida)
    throws IOException {
        File arquivo = gerenciador.buscarArquivo(nomeArquivo);

        if (arquivo == null) {
            saida.writeUTF("ERRO");
            saida.writeUTF("Arquivo não encontrado.");
            return;
        }

        saida.writeUTF("OK");
        saida.writeUTF(arquivo.getName());
        saida.writeLong(arquivo.length());

        try (FileInputStream fis = new FileInputStream(arquivo)) {
            byte[] buffer = new byte[4096];
            int bytesLidos;

            while ((bytesLidos = fis.read(buffer)) != -1) {
                saida.write(buffer, 0, bytesLidos);
            }
        }

        gerenciador.incrementarDownload(nomeArquivo);
        SwingUtilities.invokeLater(aoAtualizarTabela);
        log.accept("Arquivo baixado via TCP: " + nomeArquivo);
    }

    public void parar() {
        executando = false;
    }

```

```
}
```

8- Usar SwingWorker. Criar a Classe UDPServidor

```
package br.edu.servidor.network;

import br.edu.servidor.service.GerenciadorArquivos;

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.nio.charset.StandardCharsets;
import java.util.function.Consumer;

public class UDPServidor {

    private final int porta;
    private final GerenciadorArquivos gerenciador;
    private final Consumer<String> log;

    private boolean executando = false;

    public UDPServidor(
        int porta,
        GerenciadorArquivos gerenciador,
        Consumer<String> log
    ) {
        this.porta = porta;
        this.gerenciador = gerenciador;
        this.log = log;
    }

    public void iniciar() {
        executando = true;
    }
}
```

```

try (DatagramSocket socket = new DatagramSocket(porta)) {
    log.accept("Servidor UDP iniciado na porta " + porta);

    byte[] buffer = new byte[4096];

    while (executando) {
        DatagramPacket pacoteRecebido = new DatagramPacket(buffer,
buffer.length);

        socket.receive(pacoteRecebido);

        String mensagem = new String(
            pacoteRecebido.getData(),
            0,
            pacoteRecebido.getLength(),
            StandardCharsets.UTF_8
        );

        String resposta;

        if (mensagem.equalsIgnoreCase("DISCOVER_SERVER")) {
            resposta = "SERVIDOR_ARQUIVOS_ATIVO";
        } else if (mensagem.equalsIgnoreCase("LIST")) {
            resposta = gerenciador.gerarListaTexto();
        } else {
            resposta = "ERRO Comando UDP inválido.";
        }

        byte[] dadosResposta =
resposta.getBytes(StandardCharsets.UTF_8);

        DatagramPacket pacoteResposta = new DatagramPacket(
            dadosResposta,
            dadosResposta.length,

```

```

        pacoteRecebido.getAddress(),
        pacoteRecebido.getPort()
    );

    socket.send(pacoteResposta);

    log.accept("Mensagem UDP recebida: " + mensagem);
}

} catch (Exception e) {
    log.accept("Erro no servidor UDP: " + e.getMessage());
}
}

public void parar() {
    executando = false;
}
}

```

Execute

Arquitetura das threads

THREAD EDT (Swing)

```

|
├─ Atualiza JTable
├─ Atualiza logs
└─ Eventos da interface

```

THREAD TCP

```

|
└─ Aceita conexões

```

└─ THREAD Cliente 1
└─ THREAD Cliente 2
└─ THREAD Cliente 3
└─ ...

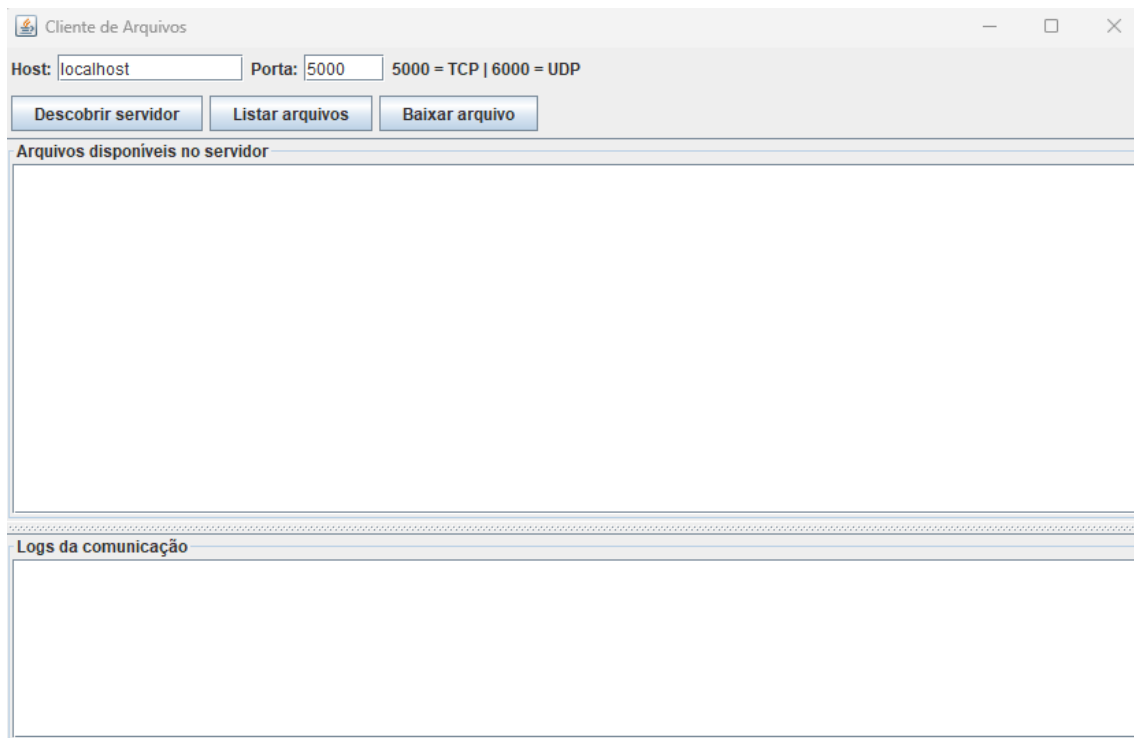
THREAD UDP

|
└─ Recebe datagramas

QUESTÕES

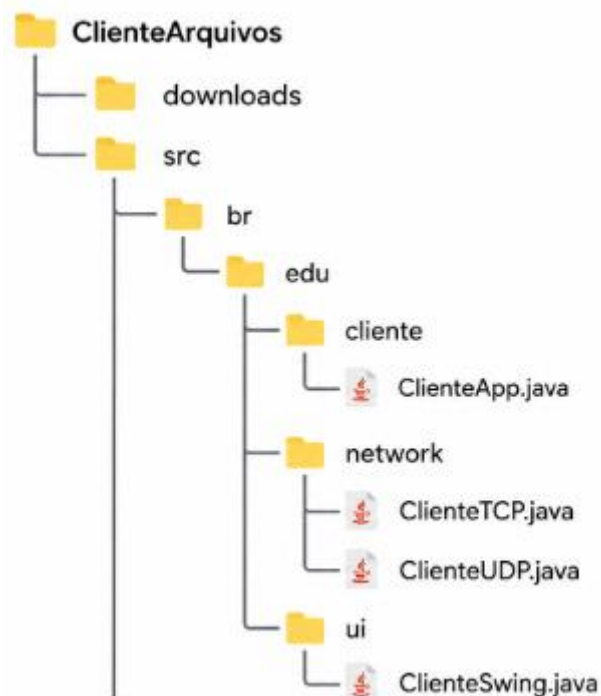
- 1- Onde existe a thread? E pra que ela é responsável?
- 2 O servidor TCP fica em loop aguardando conexões?
- 3 Há uma thread pra cada cliente? Por que?
- 4 O que correria se não fosse realizado isso?
- 5- O que aconteceria se o servidor TCP fosse executado na mesma thread da interface gráfica?

Estrutura para o CLIENTE:



Desenvolver um cliente em Java capaz de se conectar a um servidor de arquivos, listar os arquivos disponíveis e baixar arquivos selecionados.

Organização do diretório



Requisitos do cliente

O cliente deverá possuir uma interface gráfica com:

- campo para informar o host do servidor;
- campo para informar a porta;
- botão para descobrir servidor;
- botão para listar arquivos;
- botão para baixar arquivo;
- lista de arquivos disponíveis;
- área de logs.

1- Implementar a classe ClienteApp.java

```
package br.edu.cliente;

import br.edu.cliente.ui.ClienteSwing;
import javax.swing.SwingUtilities;

public class ClienteApp {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            ClienteSwing tela = new ClienteSwing();
            tela.setVisible(true);
        });
    }
}
```

2- Implementar a classe ClienteTCP.java

```
package br.edu.cliente.network;

import java.io.*;
import java.net.InetSocketAddress;
import java.net.Socket;

public class ClienteTCP {

    private final String host;
```

```

private final int porta;

public ClienteTCP(String host, int porta) {
    this.host = host;
    this.porta = porta;
}

private Socket conectar() throws IOException {
    Socket socket = new Socket();
    socket.connect(new InetSocketAddress(host, porta), 3000);
    return socket;
}

public String listarArquivos() throws Exception {
    try (
        Socket socket = conectar();
        DataOutputStream saida = new
DataOutputStream(socket.getOutputStream());
        DataInputStream entrada = new
DataInputStream(socket.getInputStream())
    ) {
        saida.writeUTF("LIST");
        saida.flush();

        String status = entrada.readUTF();

        if (!status.equalsIgnoreCase("OK")) {
            return "";
        }

        return entrada.readUTF();
    }
}

```



```

    public void baixarArquivo(String nomeArquivo, String pastaDestino)
    throws Exception {

        try (

            Socket socket = conectar();

            DataOutputStream saida = new
DataOutputStream(socket.getOutputStream());

            DataInputStream entrada = new
DataInputStream(socket.getInputStream())

        ) {

            saida.writeUTF("DOWNLOAD " + nomeArquivo);

            saida.flush();

            String status = entrada.readUTF();

            if (!status.equalsIgnoreCase("OK")) {

                String erro = entrada.readUTF();

                throw new Exception(erro);

            }

            String nomeRecebido = entrada.readUTF();

            long tamanho = entrada.readLong();

            File pasta = new File(pastaDestino);

            if (!pasta.exists()) {

                pasta.mkdirs();

            }

            File arquivoDestino = new File(pasta, nomeRecebido);

            try (FileOutputStream fos = new
FileOutputStream(arquivoDestino)) {

                byte[] buffer = new byte[4096];

                long totalRecebido = 0;

```

```

        while (totalRecebido < tamanho) {
            int bytesLidos = entrada.read(buffer);

            if (bytesLidos == -1) {
                break;
            }

            fos.write(buffer, 0, bytesLidos);
            totalRecebido += bytesLidos;
        }
    }
}
}
}
}

```

3- Implementar a classe ClienteUDP.java

```

package br.edu.cliente.network;

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.nio.charset.StandardCharsets;

public class ClienteUDP {

    private final String host;
    private final int porta;

    public ClienteUDP(String host, int porta) {
        this.host = host;
        this.porta = porta;
    }

    public String descobrirServidor() throws Exception {
        return enviarMensagem("DISCOVER_SERVER");
    }
}

```

```

    }

    public String listarArquivos() throws Exception {
        return enviarMensagem("LIST");
    }

    private String enviarMensagem(String mensagem) throws Exception {
        try (DatagramSocket socket = new DatagramSocket()) {
            socket.setSoTimeout(3000);

            byte[] dadosEnvio = mensagem.getBytes(StandardCharsets.UTF_8);

            DatagramPacket pacoteEnvio = new DatagramPacket(
                dadosEnvio,
                dadosEnvio.length,
                InetAddress.getByName(host),
                porta
            );

            socket.send(pacoteEnvio);

            byte[] buffer = new byte[4096];

            DatagramPacket pacoteResposta = new DatagramPacket(
                buffer,
                buffer.length
            );

            socket.receive(pacoteResposta);

            return new String(
                pacoteResposta.getData(),
                0,
                pacoteResposta.getLength(),
                StandardCharsets.UTF_8
            );
        }
    }
}

```

```
}
```

4- Implementar a classe ClienteSwing.java

```
package br.edu.cliente.ui;

import br.edu.cliente.network.ClienteTCP;
import br.edu.cliente.network.ClienteUDP;
import java.awt.*;
import javax.swing.*;

public class ClienteSwing extends JFrame {

    private JTextField txtHost;
    private JTextField txtPorta;

    private JButton btnDescobrir;
    private JButton btnListar;
    private JButton btnBaixar;

    private JList<String> listaArquivos;
    private DefaultListModel<String> listaModel;

    private JTextArea areaLogs;

    private static final String PASTA_DOWNLOADS = "downloads";

    public ClienteSwing() {
        setTitle("Cliente de Arquivos");
        setSize(850, 550);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        inicializarComponentes();
    }

    private void inicializarComponentes() {

        txtHost = new JTextField("localhost", 12);
```

```
txtPorta = new JTextField("5000", 5);

btnDescobrir = new JButton("Descobrir servidor");
btnListar = new JButton("Listar arquivos");
btnBaixar = new JButton("Baixar arquivo");

listaModel = new DefaultListModel<>();

listaArquivos = new JList<>(listaModel);
listaArquivos.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

areaLogs = new JTextArea();
areaLogs.setEditable(false);

JPanel painelTopo = new JPanel(new FlowLayout(FlowLayout.LEFT));

painelTopo.add(new JLabel("Host:"));
painelTopo.add(txtHost);

painelTopo.add(new JLabel("Porta:"));
painelTopo.add(txtPorta);

painelTopo.add(new JLabel("5000 = TCP | 6000 = UDP"));

JPanel painelBotoes = new JPanel(new FlowLayout(FlowLayout.LEFT));

painelBotoes.add(btnDescobrir);
painelBotoes.add(btnListar);
painelBotoes.add(btnBaixar);

JPanel painelSuperior = new JPanel(new BorderLayout());

painelSuperior.add(painelTopo, BorderLayout.NORTH);
painelSuperior.add(painelBotoes, BorderLayout.SOUTH);

JPanel painelArquivos = new JPanel(new BorderLayout());

painelArquivos.setBorder(
```

```
        BorderFactory.createTitledBorder(
            "Arquivos disponíveis no servidor"
        )
    );

    painelArquivos.add(
        new JScrollPane(listaArquivos),
        BorderLayout.CENTER
    );

    JPanel painelLogs = new JPanel(new BorderLayout());

    painelLogs.setBorder(
        BorderFactory.createTitledBorder(
            "Logs da comunicação"
        )
    );

    painelLogs.add(
        new JScrollPane(areaLogs),
        BorderLayout.CENTER
    );

    JSplitPane splitPane = new JSplitPane(
        JSplitPane.VERTICAL_SPLIT,
        painelArquivos,
        painelLogs
    );

    splitPane.setDividerLocation(280);

    add(painelSuperior, BorderLayout.NORTH);
    add(splitPane, BorderLayout.CENTER);

    btnDescobrir.addActionListener(e -> descobrirServidor());

    btnListar.addActionListener(e -> listarArquivos());
```

```
        btnBaixar.addActionListener(e -> baixarArquivoSelecionado());
    }

    private int obterPorta() {
        return Integer.parseInt(txtPorta.getText().trim());
    }

    private String obterHost() {
        return txtHost.getText().trim();
    }

    private boolean isTCP() {
        return obterPorta() == 5000;
    }

    private boolean isUDP() {
        return obterPorta() == 6000;
    }

    private ClienteTCP criarClienteTCP() {
        return new ClienteTCP(
            obterHost(),
            obterPorta()
        );
    }

    private ClienteUDP criarClienteUDP() {
        return new ClienteUDP(
            obterHost(),
            obterPorta()
        );
    }

    private void descobrirServidor() {

        SwingWorker<String, Void> worker = new SwingWorker<>() {

            @Override
```

```

protected String doInBackground() throws Exception {

    if (isTCP()) {

        adicionarLog(
            "Tentando descobrir servidor via TCP na porta 5000..."
        );

        ClienteTCP clienteTCP = criarClienteTCP();

        clienteTCP.listarArquivos();

        return "Servidor TCP ativo.";

    } else if (isUDP()) {

        adicionarLog(
            "Tentando descobrir servidor via UDP na porta 6000..."
        );

        ClienteUDP clienteUDP = criarClienteUDP();

        return clienteUDP.descobrirServidor();

    } else {

        throw new Exception(
            "Porta inválida. Use 5000 para TCP ou 6000 para UDP."
        );

    }
}

@Override
protected void done() {

    try {

        String resposta = get();
    }
}

```



```

        adicionarLog(
            "Resposta do servidor: " + resposta
        );

        JOptionPane.showMessageDialog(
            ClienteSwing.this,
            resposta
        );

    } catch (Exception e) {

        adicionarLog(
            "Servidor não encontrado: " + e.getMessage()
        );

        JOptionPane.showMessageDialog(
            ClienteSwing.this,
            "Servidor não encontrado."
        );
    }
}

};

worker.execute();
}

private void listarArquivos() {

    SwingWorker<String, Void> worker = new SwingWorker<>() {

        @Override
        protected String doInBackground() throws Exception {

            if (isTCP()) {

                adicionarLog(
                    "Listando arquivos via TCP..."
                );
            }
        }
    };
}

```

```

    );

    ClienteTCP clienteTCP = criarClienteTCP();

    return clienteTCP.listarArquivos();

} else if (isUDP()) {

    adicionarLog(
        "Listando arquivos via UDP..."
    );

    ClienteUDP clienteUDP = criarClienteUDP();

    return clienteUDP.listarArquivos();

} else {

    throw new Exception(
        "Porta inválida. Use 5000 ou 6000."
    );
}
}

@Override
protected void done() {

    try {

        String resposta = get();

        listaModel.clear();

        if (resposta == null || resposta.isBlank()) {

            adicionarLog(
                "Nenhum arquivo encontrado."
            );

```

```

        return;
    }

    String[] linhas = resposta.split("\n");

    for (String linha : linhas) {

        if (!linha.isBlank()) {

            String[] partes = linha.split(";");

            if (partes.length >= 1) {

                listaModel.addElement(
                    partes[0]
                );
            }
        }
    }

    adicionarLog(
        "Arquivos carregados na lista."
    );

} catch (Exception e) {

    adicionarLog(
        "Erro ao listar arquivos: " + e.getMessage()
    );

    JOptionPane.showMessageDialog(
        ClienteSwing.this,
        e.getMessage()
    );
}

}

};

```

```

        worker.execute();
    }

    private void baixarArquivoSelecioneado() {

        String arquivoSelecioneado =
            listaArquivos.getSelectedValue();

        if (arquivoSelecioneado == null) {

            JOptionPane.showMessageDialog(
                this,
                "Selecione um arquivo."
            );

            return;
        }

        if (!isTCP()) {

            JOptionPane.showMessageDialog(
                this,
                "Download disponível apenas na porta TCP 5000."
            );

            return;
        }

        SwingWorker<Void, Void> worker = new SwingWorker<>() {

            @Override
            protected Void doInBackground() throws Exception {

                adicionarLog(
                    "Baixando arquivo via TCP: "
                    + arquivoSelecioneado
                );
            }
        };
    }

```

```

        ClienteTCP clienteTCP =
            criarClienteTCP();

        clienteTCP.baixarArquivo(
            arquivoSelecioneado,
            PASTA_DOWNLOADS
        );

        return null;
    }

    @Override
    protected void done() {

        try {

            get();

            adicionarLog(
                "Download concluído: "
                + arquivoSelecioneado
            );

            adicionarLog(
                "Arquivo salvo na pasta downloads."
            );

            JOptionPane.showMessageDialog(
                ClienteSwing.this,
                "Download concluído."
            );

        } catch (Exception e) {

            adicionarLog(
                "Erro no download: "
                + e.getMessage()
            );
        }
    }

```

```

        );

        JOptionPane.showMessageDialog(
            ClienteSwing.this,
            "Erro no download."
        );
    }
}

};

worker.execute();
}

private void adicionarLog(String mensagem) {

    SwingUtilities.invokeLater(() -> {

        areaLogs.append(
            mensagem + "\n"
        );
    });
}
}

```

1- Aplicar um método de ordenação nos arquivos do servidor para ordená-los de forma crescente (tamanho do arquivo).

Justificar a escolha
 Apresentar o funcionamento do método
 Elaborar o diagrama de classe

2- Responder as questões propostas na atividade.